STUDENT DESK

Version 2.0 – Production Architecture Edition

Author: Vivek Boga **Year:** 2026 **Live URL:** https://studentdesk.vercel.app **Repository:** https://github.com/bogavivek15/viveks-first-project

---

Table of Contents

1. Abstract

STUDENT DESK is a production-grade, full-stack web application architected to serve as a centralized digital academic resource hub for Bachelor of Technology (B.Tech) engineering students. The platform addresses a critical gap in the Indian engineering education ecosystem: the absence of a single, organized, and accessible repository for subject-wise study materials, previous year question papers, and on-demand academic doubt resolution.

The application implements a structured navigational hierarchy spanning six engineering branches—Computer Science and Engineering (CSE), Electronics and Communication Engineering (ECE), Electrical and Electronics Engineering (EEE), Mechanical Engineering (MECH), Civil Engineering (CIVIL), and Information Technology (IT)—across four academic years and two semesters per year. This hierarchical model (Course → Year → Semester → Subject → Notes) mirrors the natural academic structure of the B.Tech curriculum, enabling intuitive resource discovery.

The platform is built using a modern technology stack comprising React 18 with TypeScript strict mode on the frontend, Supabase Cloud providing PostgreSQL database services with Row Level Security (RLS), JWT-based authentication, cloud storage for PDF files, and serverless Deno-based Edge Functions on the backend. The AI-powered doubt resolution system leverages the Groq inference platform with a three-model fallback cascade (LLaMA 3.3 70B → LLaMA 3.1 8B → Mixtral 8x7B) to deliver sub-second response times for academic queries.

Security is enforced at every layer through seven HTTP security headers including a strict Content Security Policy, database-level RLS policies, server-side rate limiting, client-side input validation using Zod runtime schema validation, honeypot-based anti-spam protection, and URL origin validation. Performance optimization is achieved through route-level code splitting with React.lazy(), Vite 5 manual chunk configuration, composite and trigram database indexing, asset immutable caching, and Terser-based dead code elimination.

The frontend is deployed on Vercel with global CDN delivery and automatic deployments from the GitHub main branch, while a GitHub Actions CI pipeline enforces ESLint linting, TypeScript strict type-checking, and production build validation on every push and pull request. The application is designed for horizontal scalability and currently serves as a live production platform accessible at studentdesk.vercel.app.

2. Introduction

The rapid digitization of academic ecosystems has created both opportunities and challenges for engineering students. While the internet provides access to an enormous volume of educational content, this abundance often results in information fragmentation—study materials are scattered across personal messaging groups, cloud storage links, and ephemeral web resources with no centralized organization. For students of the B.Tech program, where the curriculum spans four years, two semesters per year, and dozens of specialized subjects per engineering branch, the lack of systematic resource management represents a significant barrier to effective exam preparation.

STUDENT DESK was conceived as a purpose-built solution to this problem. Rather than functioning as a generic document repository, the platform is designed around the specific organizational structure of the B.Tech academic program. Every piece of content—whether a set of lecture notes or a previous year question paper—is cataloged within a five-level hierarchy: Engineering Branch, Academic Year, Semester, Subject, and Resource. This structure allows students to navigate directly to the materials they need without performing ad hoc searches across external platforms.

The Version 2.0 architecture represents a significant evolution in system design. The frontend is implemented as a Single-Page Application (SPA) using React 18 with TypeScript in strict mode, ensuring compile-time type safety across the entire codebase. The UI component layer is built on shadcn/ui, which wraps Radix UI primitives with Tailwind CSS styling, providing accessible and consistent user interface elements. Routing is handled client-side by React Router v6 with all fifteen page components lazy-loaded via React.lazy() and wrapped in Suspense boundaries.

The backend is entirely serverless, powered by Supabase Cloud. The PostgreSQL database enforces data integrity through check constraints, unique constraints, and foreign key relationships, while Row Level Security policies ensure that data access is controlled at the database layer rather than relying solely on application-level authorization checks. File storage for PDF notes uses Supabase Storage with policy-controlled access, and the AI chatbot API is implemented as a Supabase Edge Function running on the Deno runtime.

A distinguishing feature of STUDENT DESK is its AI-powered doubt resolution chatbot, available on every subject page. Students can ask academic questions in natural language and receive structured, exam-focused responses in real-time. The chatbot supports conversation memory (last six messages), language flexibility (including transliterated Telugu, referred to as "Tenglish"), and provides responses formatted in Markdown with headings, bullet points, and code blocks. The underlying inference is provided by the Groq API with a three-model fallback mechanism to ensure high availability.

This documentation provides a comprehensive technical description of the STUDENT DESK Version 2.0 architecture, covering system design, database modeling, security implementation, performance optimization, AI integration, and deployment infrastructure. It is intended to serve as both an academic reference for project evaluation and a technical guide for understanding the engineering decisions embedded in the platform.

3. Problem Statement

Engineering students pursuing the Bachelor of Technology degree encounter several systemic challenges during their academic journey, particularly during examination preparation periods. These challenges stem from the absence of a centralized, well-organized, and reliable platform for academic resource management. The following sections detail the specific problems identified through direct observation of the target user base.

3.1 Fragmentation of Study Materials

Academic resources for B.Tech students are typically distributed through informal channels. Lecture notes, handwritten materials, and reference documents are shared via WhatsApp groups, Telegram channels, Google Drive links, and email attachments. This distribution model introduces several failure modes:

- **Link Expiration:** Shared Google Drive links are often revoked or made private when the original uploader graduates, rendering the resources permanently inaccessible to subsequent batches.
- **Channel Proliferation:** Each batch, section, and study group maintains separate communication channels, leading to duplication of effort and inconsistent availability of materials across student cohorts.
- **Lack of Organization:** Files shared through messaging platforms are not categorized by subject, semester, or exam type, forcing students to manually search through message histories to locate specific resources.
- **Version Confusion:** Multiple versions of the same document may circulate across different channels, with no mechanism to identify the most current or accurate version.

3.2 Supplementary Examination Preparation Gap

Students who need to clear backlogs through supplementary examinations face a particularly acute resource shortage. Regular examination materials may not align with the supplementary examination syllabus or format, and there is typically no dedicated repository of supply-specific study materials. Students preparing for supplementary exams often study in isolation, without access to the peer networks that facilitate resource sharing during regular examination periods.

3.3 Absence of On-Demand Doubt Resolution

Traditional academic support mechanisms—faculty office hours, tutoring sessions, and peer discussions—operate within constrained time windows. Students studying during late-night hours, weekends, or holiday periods have no immediate channel for academic doubt resolution. This delay between encountering a concept difficulty and receiving clarification can significantly disrupt study continuity and reduce the effectiveness of self-directed learning sessions.

3.4 No Unified Academic Platform

Each engineering branch, academic year, and student batch maintains its own informal resource management system. There is no single platform that provides structured access to materials across all branches and semesters. This fragmentation means that institutional knowledge is repeatedly lost as student cohorts graduate, and incoming students must reconstruct resource collections from scratch.

4. Proposed Solution

STUDENT DESK addresses each identified problem through purpose-built features designed around the specific needs of B.Tech engineering students. The solution is architected as a production web application that is accessible 24/7 from any device with an internet connection.

4.1 Centralized Resource Organization

The platform implements a five-level navigational hierarchy that directly mirrors the B.Tech academic structure:

```
Engineering Branch → Academic Year → Semester → Subject → Resources
```

All study materials—lecture notes, reference documents, and previous year question papers—are uploaded with structured metadata including course association, year, semester, subject, resource type (notes or question papers), and exam type (regular, supply, or both). This metadata-driven organization eliminates the need for ad hoc searching and guarantees consistent resource discoverability regardless of when or by whom the material was uploaded.

4.2 Examination-Specific Resource Tagging

Every uploaded resource is tagged with an examination type classification:

- **Regular:** Materials relevant to regular semester examinations.
- **Supply:** Materials specifically curated for supplementary examinations.
- **Both:** Materials applicable to both examination types.

This tagging system allows students preparing for supplementary examinations to filter resources directly relevant to their needs, eliminating the guesswork involved in identifying supply-appropriate materials from generic resource collections.

4.3 AI-Powered Instant Doubt Resolution

An AI chatbot powered by the Groq inference platform is integrated into every subject page. The chatbot provides:

- **Context-Aware Responses:** The chatbot receives the current subject name and topic as context, ensuring that responses are relevant to the material the student is currently studying.
- **Conversation Memory:** The system maintains the last six messages of conversation history, allowing follow-up questions that build on previous responses.
- **Exam-Focused Formatting:** Responses are structured with bold headings, bullet points, definitions, key points, and exam-important markers to facilitate efficient study and revision.
- **Multilingual Support:** The chatbot can respond in regional languages upon request, including transliterated Telugu using English letters (Tenglish), accommodating the linguistic preferences of the target user base.
- **24/7 Availability:** As a serverless AI system, the chatbot is available at any time, eliminating the dependency on faculty availability for doubt resolution.

4.4 Unified Platform with Administrative Control

The platform provides a dedicated administrative panel for content management, enabling authorized administrators to:

- Upload PDF study materials with structured metadata.
- Manage the subject catalog (add, edit, delete subjects with course, year, and semester assignment).
- Manage engineering branches (add, edit, delete courses).

- View and manage contact form submissions from students.

This administrative layer ensures that the platform can be maintained and expanded without requiring technical intervention, enabling sustainable operation by faculty or student coordinators.

5. Technology Stack

The technology stack for STUDENT DESK Version 2.0 was selected based on criteria of type safety, developer productivity, runtime performance, security by default, and production-readiness at zero operational cost. Each technology serves a specific architectural purpose, and the stack as a whole is designed to minimize operational complexity while maximizing reliability.

5.1 Frontend Technologies

Technology	Version	Purpose
React	18.3	Component-based UI library for building the Single-Page Application. Utilizes functional components, hooks, and Strict Mode for detecting potential problems during development.
TypeScript	5.8	Statically-typed superset of JavaScript with strict mode enabled. Provides compile-time type checking across all source files, eliminating an entire class of runtime errors related to type mismatches.
Vite	5.4	Next-generation frontend build tool and development server. Provides sub-second Hot Module Replacement (HMR) during development and optimized Rollup-based production builds with manual chunk splitting.
Tailwind CSS	3.4	Utility-first CSS framework that enables rapid UI development without writing custom stylesheets. Combined with the Tailwind Typography plugin for prose content styling.
shadcn/ui	Latest	Pre-built, accessible UI component collection built on Radix UI primitives. Provides components such as Dialog, DropdownMenu, Tabs, Select, ScrollArea, Form, Table, and Card with full keyboard navigation and ARIA compliance.
React Router	6.30	Client-side routing library with declarative route definitions, nested routes, and parameterized URL segments. Configured with v7 compatibility flags for forward-compatible migration.

Technology	Version	Purpose
React Hook Form	7.61	Performant form state management library that minimizes re-renders by utilizing uncontrolled components internally. Integrated with Zod for schema-based validation.
Zod	3.25	TypeScript-first runtime schema validation library. Used for validating all form inputs (registration, login, contact, profile, upload) with type-safe error messages.
React Markdown	10.1	Markdown-to-React renderer used to display AI chatbot responses with proper formatting including headings, lists, code blocks, and bold text.
react-helmet-async	2.0	Server-safe head management library for setting dynamic per-page meta tags including title, description, Open Graph, Twitter Card, and canonical URL tags for SEO optimization.
@tanstack/react-query	5.83	Data synchronization library configured with a stale time of five minutes and garbage collection after thirty minutes. Provides request deduplication and automatic retry with centralized cache management.
Lucide React	0.462	Icon library providing SVG-based icons used throughout the UI for navigation, actions, and visual indicators.
Sonner	1.7	Toast notification library for displaying success, error, and informational messages to users.

5.2 Backend Technologies

Technology	Purpose
Supabase Cloud	Backend-as-a-Service platform providing managed PostgreSQL database, JWT-based authentication, cloud file storage, Row Level Security policies, and serverless Edge Functions. Eliminates the need for custom server infrastructure.

Technology	Purpose
PostgreSQL	Enterprise-grade relational database system. All data is stored in a normalized schema with six primary tables, enforced through check constraints, unique constraints, and foreign key relationships. Accessed through the Supabase JavaScript client library with automatic RLS enforcement.
Supabase Auth	Authentication service providing email/password credential management, JWT token issuance, session management, email verification, and password reset flows. Tokens are automatically attached to all Supabase client requests.
Supabase Storage	Object storage service for PDF files. Files are organized in the "notes" bucket with RLS policies controlling upload (admin only) and download (public read) access.
Supabase Edge Functions	Serverless function runtime based on Deno. Hosts the AI chatbot API endpoint at `ask-gemini`, which handles JWT authentication, rate limiting, input validation, and Groq API communication. Deployed globally on Supabase infrastructure.

5.3 AI and LLM Infrastructure

Technology	Purpose
Groq API	Ultra-fast Large Language Model inference provider. Selected for sub-second inference latency on models with up to 70 billion parameters, significantly outperforming traditional cloud LLM providers for real-time chatbot interactions.
LLaMA 3.3 70B Versatile	Primary AI model. A 70-billion parameter model from Meta, providing the highest quality responses for complex academic queries. Used as the first-choice model in the fallback cascade.
LLaMA 3.1 8B Instant	First fallback model. An 8-billion parameter model optimized for speed. Activated when the primary model is rate-limited or times out, providing faster responses at slightly reduced depth.
Mixtral 8x7B 32768	Second fallback model. A Mixture-of-Experts model with 32K context window. Serves as the final fallback option, ensuring that the chatbot can still respond even when both LLaMA models are unavailable.

5.4 Hosting and DevOps

Technology	Purpose
Vercel	Frontend hosting platform with global CDN distribution, automatic SSL certificate management, and zero-configuration deployments from the GitHub main branch. Configured with SPA rewrites and security headers via `vercel.json`.
Supabase Cloud	Hosts the PostgreSQL database, authentication service, storage service, and Edge Functions on managed infrastructure with automatic backups and monitoring.
GitHub Actions	Continuous Integration pipeline that executes ESLint linting, TypeScript strict type-checking, and Vite production build validation on every push to the main branch and every pull request.
Terser	JavaScript minification engine used during production builds. Configured to strip all `console.log` statements and debugger statements from production bundles, preventing information leakage and reducing bundle size.

6. System Architecture

6.1 High-Level Architecture Overview

STUDENT DESK follows a client-server architecture where the frontend is a statically-hosted Single-Page Application (SPA) that communicates with serverless backend services over HTTPS. There is no traditional application server; all server-side logic is handled by Supabase managed services and Edge Functions.

The architecture consists of four primary layers:

1. **Presentation Layer (Client):** A React 18 SPA served from Vercel's global CDN. The client handles all UI rendering, client-side routing, form validation, and state management. It communicates with the backend exclusively through the Supabase JavaScript client library and the Edge Function invocation API.
2. **API Gateway Layer (Supabase):** The Supabase platform acts as the unified API gateway. The JavaScript client library provides typed methods for database queries (auto-transpiled to PostgREST API calls), authentication operations, and storage file management. Row Level Security policies are evaluated at this layer, ensuring that every database operation is authorized regardless of the client implementation.
3. **Serverless Compute Layer (Edge Functions):** The AI chatbot functionality is implemented as a Supabase Edge Function running on the Deno runtime. This function handles its own authentication verification, rate limiting, input validation, and external API communication with the Groq inference service. The Edge Function is deployed globally on Supabase infrastructure and scales automatically with demand.
4. **Data Layer (PostgreSQL + Storage):** The PostgreSQL database stores all structured application data—user profiles, roles, courses, subjects, note metadata, and contact messages. PDF files are stored in Supabase Storage with public read access and admin-only write access. The data layer enforces integrity through constraints, triggers, and RLS policies.

6.2 Client-Side Architecture

The React frontend is structured into five module categories, each with a distinct responsibility:

Pages (15 components): Each page represents a distinct route in the application and is loaded on-demand using `React.lazy()`. Pages include Home, Register, Login, ForgotPassword, UpdatePassword, About, Contact, Dashboard, Profile, CourseYears, YearSemesters, SemesterSubjects, SubjectNotes, AdminPanel, and NotFound.

Components: Shared UI elements used across multiple pages. This includes the Navbar (with sticky positioning, dark mode toggle, and responsive mobile menu via Sheet), Footer, ChatBot (floating widget with Markdown rendering), ErrorBoundary (class component wrapping the entire application), PageMeta (dynamic SEO tags), and ProtectedRoute/AdminRoute (authentication and authorization guards).

Contexts: Global state providers implemented using the React Context API. The AuthContext manages user session state, admin role checking, and provides authentication methods (signUp, signIn, signOut, resetPassword, updatePassword). The ThemeContext manages the dark mode system with support for light, dark, and system preferences, persisted to localStorage.

Hooks: Custom React hooks including `use-mobile` for responsive breakpoint detection and `use-toast` for toast notification management.

Integrations: The Supabase client initialization and auto-generated database type definitions are encapsulated in the integrations module, providing a single point of configuration for all backend communication.

The component tree is wrapped in a multi-layer provider hierarchy:

```
<ErrorBoundary>
  <HelmetProvider>
    <ThemeProvider>
      <QueryClientProvider>
        <TooltipProvider>
          <BrowserRouter>
            <AuthProvider>
              {/* Application UI */}
            </AuthProvider>
          </BrowserRouter>
        </TooltipProvider>
      </QueryClientProvider>
    </ThemeProvider>
  </HelmetProvider>
</ErrorBoundary>
```

This hierarchy ensures that error boundaries capture all rendering failures, SEO meta tags are managed globally, theming is available to all components, data caching is centralized, tooltips render in the correct stacking context, routing is available before authentication state is resolved, and authentication state is accessible to all routed components.

6.3 Server-Side Architecture

The server-side architecture is entirely serverless and consists of three managed services:

Supabase PostgreSQL: Hosts the application database with six primary tables (profiles, user_roles, courses, subjects, notes, contact_messages), three database functions (has_role, handle_new_user, update_updated_at), and three triggers that automate profile creation, role assignment, and timestamp management. The database is accessed through Supabase's PostgREST layer, which automatically translates REST API calls into SQL queries with RLS policy enforcement.

Supabase Storage: Provides a managed object storage service with a dedicated "notes" bucket for PDF files. Storage access is controlled by RLS policies: any user can read (download) files, but only administrators can upload, update, or delete files. File URLs are generated by Supabase and include the project reference, ensuring that download links are always valid and consistent.

Supabase Edge Function (ask-gemini): A single serverless function written in TypeScript for the Deno runtime. This function serves as the API endpoint for the AI chatbot. It performs JWT authentication by creating a Supabase client with the caller's authorization header and verifying the user identity. It implements in-memory rate limiting using a Map data structure, enforcing a minimum three-second interval between requests per user. It validates incoming messages for length (maximum 2000 characters) and format, constructs a system prompt incorporating the subject context, and calls the Groq API with a three-model fallback cascade.

6.4 End-to-End Data Flow

The following describes the complete data flow for the five primary user interactions:

Flow 1 — Application Loading: The user's browser requests the SPA from Vercel's CDN. Vercel serves the static HTML, CSS, and JavaScript bundles with security headers (CSP, HSTS, X-Frame-Options). The React application mounts, checks for an existing Supabase session in localStorage, and restores the authentication state if a valid JWT is found. The AuthContext subscribes to Supabase's `onAuthStateChange` event for real-time session management.

Flow 2 — User Registration: The user submits the registration form. React Hook Form validates the inputs against a Zod schema (name 1–100 chars, email valid format, password 8–100 chars, passwords match). The validated data is sent to Supabase Auth via `supabase.auth.signUp()` with the user's name and optional branch stored in `raw_user_meta_data`. Supabase creates the auth.users record and fires the

`on_auth_user_created` trigger, which executes `handle_new_user()` to create a profiles record and assign the 'student' role in user_roles. A verification email is sent to the user.

Flow 3 — Course Browsing and Note Access: The authenticated user navigates through the Dashboard → Course → Year → Semester → Subject hierarchy. Each navigation step triggers a Supabase query filtered by the relevant parameters (course_id, year, semester, subject_id). These queries are optimized by the composite index `idx_subjects_course_year_sem` and the individual FK indexes. The SubjectNotes page displays available PDF resources with View and Download actions, plus the AI ChatBot floating widget.

Flow 4 — AI Chatbot Interaction: The user types a question in the chatbot input. The client enforces a five-second cooldown between messages. The message (with subject context and last six conversation messages) is sent to the Edge Function via `supabase.functions.invoke('ask-gemini')`. The Edge Function verifies the JWT, checks the server-side rate limit (three-second minimum interval), validates the message length, constructs the system prompt with nine behavior rules, and calls the Groq API. If the primary model (LLaMA 3.3 70B) fails or times out (10-second AbortController), the function automatically retries with the fallback models. The AI response is returned as Markdown and rendered in the chat interface using ReactMarkdown with custom component overrides.

Flow 5 — Admin Content Management: An administrator accesses the Admin Panel (protected by AdminRoute which checks isAdmin status). The Upload Notes tab provides a form to select course, year, semester, subject, resource type, and exam type, then upload a PDF file (maximum 10MB). The file is first uploaded to Supabase Storage in the "notes" bucket, and the resulting URL is stored alongside the metadata in the notes table. The Manage Subjects and Manage Courses tabs provide full CRUD operations on the academic catalog.

7. Database Design

The database is implemented in PostgreSQL, hosted on Supabase Cloud, and managed through version-controlled SQL migration files. The schema is designed for referential integrity, query performance, and security through Row Level Security.

7.1 Entity-Relationship Model

The database contains six primary application tables and one system table (auth.users), connected through foreign key relationships:

```
auth.users (Supabase managed)

       profiles (1:1 - extended user information)
            Fields: id, name, email, branch, created_at, updated_at

       user_roles (1:N - role assignments)
            Fields: id, user_id, role, created_at

       notes.uploaded_by (1:N - content authorship)

courses (standalone entity - engineering branches)
           Fields: id, name, short_name, description, created_at

       subjects (1:N - academic subjects per course)
            Fields: id, name, code, course_id, year, semester, created_at

       notes (1:N - study materials per course)
            Fields: id, title, description, course_id, subject_id,
                    year, semester, resource_type, exam_type,
                    file_url, file_name, uploaded_by, created_at, updated_at

contact_messages (standalone entity - user inquiries)
       Fields: id, name, email, message, is_read, created_at, updated_at
```

The `auth.users` table is managed internally by Supabase Auth and contains core authentication data (email, encrypted password, metadata). The `profiles` table extends this with application-specific user information. The `user_roles` table implements a flexible role assignment model that supports multiple roles per user, though the current implementation uses two roles: `admin` and `student`.

7.2 Table Schemas

profiles

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY, REFERENCES auth.users(id) ON DELETE CASCADE	User identifier, matches the Supabase auth user ID
name	TEXT	NOT NULL	User's display name, extracted from registration metadata

Column	Type	Constraints	Description
email	TEXT	NOT NULL	User's email address, synchronized with auth.users
branch	TEXT	NULLABLE	Engineering branch (CSE, ECE, EEE, MECH, CIVIL, IT)
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Profile creation timestamp
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Last modification timestamp (auto-updated by trigger)

user_roles

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique role assignment identifier
user_id	UUID	NOT NULL, REFERENCES auth.users(id) ON DELETE CASCADE	Associated user
role	app_role	NOT NULL	Assigned role (admin or student)
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Role assignment timestamp
—	—	UNIQUE(user_id, role)	Prevents duplicate role assignments

courses

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Course identifier
name	TEXT	NOT NULL	Full course name (e.g., "Computer Science and Engineering")
short_name	TEXT	NOT NULL, UNIQUE	Abbreviated name (e.g., "CSE")
description	TEXT	NULLABLE	Course description text
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Creation timestamp

subjects

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Subject identifier
name	TEXT	NOT NULL	Subject name (e.g., “Data Structures”)
code	TEXT	NOT NULL	Subject code (e.g., “CS301”)
course_id	UUID	NOT NULL, REFERENCES courses(id) ON DELETE CASCADE	Parent course
year	INTEGER	NOT NULL, CHECK (year >= 1 AND year <= 4)	Academic year (1 through 4)
semester	INTEGER	NOT NULL, CHECK (semester >= 1 AND semester <= 2)	Semester (1 or 2)
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Creation timestamp
—	—	UNIQUE(code, course_id)	Subject code unique within each course

notes

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Note identifier
title	TEXT	NOT NULL, CHECK (char_length(title) <= 500)	Note title with maximum length constraint
description	TEXT	NULLABLE	Optional description of the note content
course_id	UUID	NOT NULL, REFERENCES courses(id) ON DELETE CASCADE	Associated course
subject_id	UUID	NOT NULL, REFERENCES subjects(id) ON DELETE CASCADE	Associated subject
year	INTEGER	NOT NULL, CHECK (year >= 1 AND year <= 4)	Academic year
semester	INTEGER	NOT NULL, CHECK (semester >= 1 AND semester <= 2)	Semester
resource_type	resource_type	NOT NULL, DEFAULT ‘notes’	Type: notes or question_papers
exam_type	exam_type	NOT NULL, DEFAULT ‘both’	Exam relevance: regular, supply, or both

Column	Type	Constraints	Description
file_url	TEXT	NOT NULL	Supabase Storage URL for the PDF file
file_name	TEXT	NOT NULL	Original filename of the uploaded PDF
uploaded_by	UUID	REFERENCES auth.users(id)	Uploader identity (nullable for system uploads)
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Upload timestamp
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Last modification timestamp (auto-updated by trigger)

contact_messages

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Message identifier
name	TEXT	NOT NULL	Sender's name
email	TEXT	NOT NULL	Sender's email address
message	TEXT	NOT NULL, CHECK (char_length(message) <= 2000)	Message content with maximum length
is_read	BOOLEAN	DEFAULT FALSE	Read status for admin management
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Submission timestamp
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Last modification timestamp

7.3 Enumerations and Custom Types

The database defines three custom enumeration types to enforce domain-specific value constraints:

```
CREATE TYPE public.app_role AS ENUM ('admin', 'student');
CREATE TYPE public.exam_type AS ENUM ('regular', 'supply', 'both');
CREATE TYPE public.resource_type AS ENUM ('notes', 'question_papers');
```

These enumerations ensure that only valid values can be stored in the respective columns, providing data integrity at the database level without requiring application-layer validation for stored data.

7.4 Constraints and Data Integrity

The database enforces integrity through multiple constraint types:

- **Primary Key Constraints:** Every table uses a UUID primary key generated by `gen_random_uuid()`, except for `profiles` which reuses the auth.users UUID to establish a one-to-one relationship.
- **Foreign Key Constraints:** All inter-table relationships are enforced through foreign keys with `ON DELETE CASCADE`, ensuring that dependent records are automatically removed when parent records are deleted.

- **Unique Constraints:** `courses.short_name` is globally unique; `subjects(code, course_id)` ensures subject codes are unique within each course; `user_roles(user_id, role)` prevents duplicate role assignments.
- **Check Constraints:** `subjects.year` and `notes.year` are constrained to the range 1–4; `subjects.semester` and `notes.semester` are constrained to the range 1–2; `notes.title` is limited to 500 characters; `contact_messages.message` is limited to 2000 characters.
- **Not Null Constraints:** All required fields are marked NOT NULL, preventing incomplete data insertion.

7.5 Indexes and Query Optimization

The database includes eight purpose-built indexes designed to optimize the most frequent query patterns:

```
-- Composite index for the primary navigation query
-- Used by SemesterSubjects page: WHERE course_id = ? AND year = ? AND semester = ?
CREATE INDEX idx_subjects_course_year_sem
  ON public.subjects (course_id, year, semester);

-- Foreign key lookup indexes for notes
CREATE INDEX idx_notes_subject_id ON public.notes (subject_id);
CREATE INDEX idx_notes_course_id ON public.notes (course_id);
CREATE INDEX idx_notes_uploaded_by ON public.notes (uploaded_by);

-- RLS policy performance: has_role() function called on every write operation
CREATE INDEX idx_user_roles_user_id ON public.user_roles (user_id);

-- Descending index for admin message listing
CREATE INDEX idx_contact_messages_created_at
  ON public.contact_messages (created_at DESC);

-- Trigram GIN indexes for ILIKE text search on Dashboard
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX idx_subjects_name_trgm
  ON public.subjects USING gin (name gin_trgm_ops);
CREATE INDEX idx_subjects_code_trgm
  ON public.subjects USING gin (code gin_trgm_ops);
```

The composite index `idx_subjects_course_year_sem` is the most critical index in the system, as it directly supports the primary navigation query that powers the SemesterSubjects page. Without this index, the query would require a full table scan on the subjects table for every navigation request.

The trigram GIN indexes (`idx_subjects_name_trgm` and `idx_subjects_code_trgm`) enable efficient pattern matching for the Dashboard search feature, which uses ILIKE queries with wildcard prefixes. The `pg_trgm` extension decomposes text into three-character sequences and builds an inverted index, allowing PostgreSQL to use the index for `ILIKE '%search_term%'` queries that would otherwise bypass standard B-tree indexes.

7.6 Triggers and Automated Functions

Three database triggers automate critical operations:

handle_new_user() — Profile and Role Auto-Creation:

```
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
```

```
BEGIN
  INSERT INTO public.profiles (id, email, name, branch)
  VALUES (
    NEW.id,
    NEW.email,
    COALESCE(NEW.raw_user_meta_data ->> 'name', ''),
    NEW.raw_user_meta_data ->> 'branch'
  );
  RETURN NEW;
END;
$$;

CREATE TRIGGER on_auth_user_created
  AFTER INSERT ON auth.users
  FOR EACH ROW
  EXECUTE FUNCTION public.handle_new_user();
```

This trigger fires after every new user registration in Supabase Auth. It automatically creates a corresponding profiles record with the user's name and branch extracted from the registration metadata, and assigns the 'student' role via the user_roles table. The `SECURITY DEFINER` attribute ensures the function executes with the privileges of its creator (the database owner), bypassing RLS policies that would otherwise prevent the new user from inserting into profiles before their session is fully established.

update_updated_at() — Automatic Timestamp Management:

```
CREATE OR REPLACE FUNCTION public.update_updated_at()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
  NEW.updated_at = NOW();
  RETURN NEW;
END;
$$;
```

This function is attached to the `profiles` and `notes` tables via BEFORE UPDATE triggers, ensuring that the `updated_at` column always reflects the most recent modification timestamp without requiring application code to explicitly set it.

has_role() — Security Definer Role Check:

```
CREATE OR REPLACE FUNCTION public.has_role(_user_id UUID, _role app_role)
RETURNS BOOLEAN
LANGUAGE SQL
STABLE
SECURITY DEFINER
SET search_path = public
AS $$
  SELECT EXISTS (
    SELECT 1
    FROM public.user_roles
    WHERE user_id = _user_id AND role = _role
  )
$$;
```

This function is used extensively in RLS policies to determine whether the current user holds a specific role (typically 'admin'). It is declared as `SECURITY DEFINER` with a pinned `search_path` to prevent search path

injection attacks and to ensure the function can read the user_roles table regardless of the calling user's permissions.

8. Authentication and Authorization

Authentication and authorization in STUDENT DESK are implemented as a multi-layered system spanning the frontend application, the Supabase Auth service, and the PostgreSQL database. This defense-in-depth approach ensures that access control is enforced regardless of how the backend is accessed.

8.1 Authentication Flow

User authentication follows the standard email/password credential flow managed by Supabase Auth:

Registration Process:

1. The user fills out the registration form with name, email, password, password confirmation, and optional engineering branch.
2. The frontend validates all inputs against a Zod schema: name (1–100 characters), email (valid format, 1–255 characters), password (8–100 characters), and confirmation match.
3. The validated data is sent to Supabase Auth via `supabase.auth.signUp()`. The name and branch are passed as `raw_user_meta_data` options.
4. Supabase creates the `auth.users` record, hashes the password, and sends a verification email with a redirect URL to the application origin.
5. The `on_auth_user_created` trigger fires, executing `handle_new_user()` to create the profiles record and assign the 'student' role.
6. The user receives a JWT token upon successful authentication.

Sign-In Process:

1. The user submits their email and password on the login form.
2. Supabase Auth verifies the credentials against the stored hash.
3. Upon successful verification, Supabase issues a JWT containing the user's ID, email, and session metadata.
4. The AuthContext stores the session and user objects in React state and subscribes to `onAuthStateChange` for real-time session updates.
5. The AuthContext asynchronously checks the user's admin status by querying the `user_roles` table.

Password Reset Flow:

1. The user requests a password reset on the ForgotPassword page by entering their email.
2. Supabase sends a password reset email containing a secure link that redirects to `/update-password`.
3. When the user clicks the link, Supabase Auth fires a `PASSWORD_RECOVERY` event, which the AuthContext intercepts to navigate the user to the UpdatePassword page.
4. The user enters a new password, which is submitted via `supabase.auth.updateUser()`.

8.2 Role-Based Access Control

The application implements a two-role access control model:

Student Role: - Assigned automatically upon registration via the `handle_new_user()` trigger. - Grants read access to courses, subjects, and notes. - Grants download access to PDF files in Supabase Storage. - Grants write access to the contact form (insert into contact_messages). - Grants read/update access to the user's own profile. - Grants access to the AI chatbot via the Edge Function.

Admin Role: - Must be manually assigned by inserting a record into the `user_roles` table. - Inherits all student capabilities. - Additionally grants full CRUD access to courses, subjects, and notes tables. - Grants upload/update/delete access to the Storage "notes" bucket. - Grants read/update/delete access to all contact_messages. - Grants access to the Admin Panel UI.

Role checking on the frontend is performed by the AuthContext, which queries the `user_roles` table for an admin role matching the current user's ID. This check is performed on initial session load and whenever the authentication state changes.

8.3 Row Level Security Enforcement

Row Level Security (RLS) is enabled on all six application tables and the Storage objects table. RLS policies define predicates that are evaluated for every database operation, regardless of whether the request originates from the frontend application, a direct API call, or any other client.

The RLS policy structure follows a consistent pattern:

- **SELECT operations** on public-facing data (courses, subjects, notes) are unrestricted, allowing any user (including anonymous) to read the academic catalog.
- **SELECT operations** on user-specific data (profiles, user_roles) are restricted to the owning user via `auth.uid() = id` or `auth.uid() = user_id` predicates.
- **INSERT, UPDATE, DELETE operations** on content tables (courses, subjects, notes) are restricted to admin users via the `has_role(auth.uid(), 'admin')` function.
- **INSERT operations** on contact_messages are unrestricted (anyone can submit the contact form), while read/update/delete operations are admin-only.
- **Storage policies** mirror the table policies: public read, admin-only write.

8.4 Route Guards and Client-Side Protection

Two client-side route guard components enforce authentication and authorization at the routing layer:

ProtectedRoute: Wraps routes that require authentication (Dashboard, Profile). If the user is not authenticated, it renders a `<Navigate to="/login">` redirect. While the authentication state is loading, it displays a spinner to prevent flash of unauthorized content.

```
export const ProtectedRoute = ({ children }: ProtectedRouteProps) => {
  const { user, loading } = useAuth();
  if (loading) return <Spinner />;
  if (!user) return <Navigate to="/login" replace />;
  return <>{children}</>;
};
```

AdminRoute: Wraps routes that require admin authorization (AdminPanel). If the user is not authenticated or does not have admin status, it renders a `<Navigate to="/dashboard">` redirect.

```
export const AdminRoute = ({ children }: ProtectedRouteProps) => {
  const { user, isAdmin, loading } = useAuth();
  if (loading) return <Spinner />;
  if (!user || !isAdmin) return <Navigate to="/dashboard" replace />;
  return <>{children}</>;
};
```

These guards provide a user-friendly experience by preventing unauthorized users from seeing protected UI elements. However, they are not a security boundary—actual data security is enforced by the RLS policies at the database layer, ensuring that even if a client-side guard is bypassed, unauthorized data access is prevented.

8.5 Password Policy

Password requirements follow the NIST Special Publication 800-63B guidelines:

- **Minimum length:** 8 characters (enforced by Zod schema on the frontend and Supabase Auth on the backend).
- **Maximum length:** 100 characters (prevents excessively long password hashing operations).

- **No composition rules:** Consistent with NIST recommendations, the system does not mandate uppercase, lowercase, digit, or special character requirements, as research has shown that composition rules reduce password entropy by encouraging predictable patterns.
- **Confirmation matching:** The registration form requires password confirmation to prevent typos.

9. AI Integration

The AI-powered doubt resolution system is one of the distinguishing features of STUDENT DESK. It provides students with instant, context-aware academic assistance on every subject page, functioning as a 24/7 virtual teaching assistant.

9.1 Chatbot Architecture

The chatbot system consists of two components: the ChatBot React component on the frontend and the `ask-gemini` Edge Function on the backend.

Frontend Component (ChatBot.tsx): The chatbot renders as a floating widget in the bottom-right corner of every SubjectNotes page. It is implemented as a React functional component with the following state management:

- `isOpen` — Controls the visibility of the chat panel (toggled by the floating action button).
- `messages` — An array of message objects, each containing a `role` ('user' or 'assistant') and `content` string.
- `input` — The current text in the message input field.
- `isLoading` — Indicates whether an AI response is being awaited.
- `isCooldown` — Enforces a five-second client-side delay between messages.

The component initializes with a welcome message that includes the subject name, providing immediate context to the user. Messages are rendered using ReactMarkdown with custom component overrides for headings, lists, paragraphs, bold text, and code blocks, ensuring that AI responses are displayed with proper formatting.

UI Design Details: The chat panel uses responsive dimensions: `w-[calc(100vw-2rem)]` on mobile with a `max-w-[350px]` cap, and `h-[60vh]` on mobile to account for the on-screen keyboard, scaling to a fixed `sm:h-[500px]` and `sm:w-[350px]` on desktop. The panel features a gradient glow animation effect, backdrop blur, and smooth slide-in animation.

9.2 AI Request Flow

When a user sends a message, the following sequence of operations occurs:

1. **Client-Side Validation:** The input is checked for non-empty content and a maximum length of 2000 characters. If the five-second cooldown is active, the send action is blocked.
2. **Message Preparation:** The user's message is appended to the messages array, and the input field is cleared. A loading indicator is displayed in the chat panel.
3. **Cooldown Activation:** A five-second cooldown timer starts, disabling the input field and showing a clock icon on the send button.
4. **History Extraction:** The last six messages (excluding the initial welcome message) are extracted as conversation history.
5. **Edge Function Invocation:** The message, subject context string (e.g., "Subject: Data Structures, Specific Topic: Trees"), and conversation history are sent to the `ask-gemini` Edge Function via `supabase.functions.invoke()`.
6. **Server-Side Processing (Edge Function):**
   - CORS preflight is handled for OPTIONS requests.
   - The Authorization header is extracted and used to create a Supabase client that verifies the JWT.
   - If the user is not authenticated, a 401 response is returned.

- The server-side rate limiter checks if at least three seconds have elapsed since the user's last request. If not, a 429 response is returned.
- The request body is parsed and the message is validated for content and length.
- The system prompt is constructed with nine behavior rules and the subject context.
- The Groq API is called with the complete message sequence (system prompt + conversation history + current message).

7. **Response Delivery:** The AI-generated response is returned as a JSON object containing the `reply` field. The ChatBot component appends the response to the messages array and renders it as Markdown.

9.3 Multi-Model Fallback Cascade

The Edge Function implements a three-model fallback mechanism to maximize chatbot availability. LLM inference providers impose rate limits per model, and high-traffic periods can lead to temporary model unavailability. The fallback cascade ensures that users receive a response even when the primary model is temporarily throttled.

```
Request → LLaMA 3.3 70B Versatile

                Success → Return response

                429 (Rate Limited) → Continue to next model

                Timeout (10s) → Continue to next model

                Error → Continue to next model


          LLaMA 3.1 8B Instant

                Success → Return response

                429 / Timeout / Error → Continue to next model


          Mixtral 8x7B 32768

                Success → Return response

                All Failed → Return error message (HTTP 500)
```

Each model call is wrapped in an AbortController with a ten-second timeout. This prevents a single slow model from blocking the entire request. The fallback is transparent to the user—they receive a response regardless of which model generated it.

The inference parameters are consistent across all models:

```
{
  "temperature": 0.3,
  "max_tokens": 2048
}
```

A low temperature of 0.3 is used to produce focused, factual academic responses rather than creative or divergent outputs. The 2048 max_tokens limit allows for comprehensive answers while preventing excessively long responses that would degrade the chat interface experience.

9.4 System Prompt Engineering

The system prompt defines nine distinct behavior rules that govern the chatbot's response patterns:

1. **Greetings:** When the user sends a simple greeting (hi, hello, hey) without a question, the chatbot responds with a brief 1–2 line friendly greeting that references the current subject.
2. **Personal Messages:** If the user shares their name or personal information, the chatbot acknowledges it warmly rather than redirecting to academic content.
3. **Conversational Messages:** Casual or conversational messages receive natural, human-like responses rather than formulaic academic redirects.
4. **Simple Questions:** Standard academic questions receive concise, exam-focused answers of approximately 200–300 words, formatted with bold headings and bullet points.
5. **Detailed Requests:** When the user explicitly requests a detailed or in-depth explanation, the chatbot generates a comprehensive 500–1000 word response structured with Definition, Explanation, Key Points, Examples, Advantages/Disadvantages, and Exam-Important Points sections.
6. **Language Requests:** The chatbot can respond in any requested language, including transliterated Telugu using English letters (Tenglish), which is a common communication preference among the target user demographic in the Telugu-speaking regions of India.
7. **Off-Topic Handling:** Non-academic questions are politely redirected to the subject context, while maintaining conversational naturalness.
8. **Markdown Formatting:** All responses use proper Markdown formatting for readability.
9. **Conversation Memory:** The chatbot is instructed to use conversation history for contextual continuity.

The subject context is dynamically injected into the system prompt based on the current SubjectNotes page:

```
Subject Context: Subject: Data Structures, Specific Topic: Binary Trees
```

9.5 Conversation Memory

The chatbot maintains conversation context by sending the last six messages as `history` in each API request. This allows the AI model to reference previous exchanges when generating new responses. For example, if a student asks "Explain binary search trees" followed by "What are the advantages?", the model understands that "the advantages" refers to binary search trees without requiring the student to repeat the topic.

The history is trimmed to six messages (three user messages and three assistant responses) to balance context quality against token consumption and latency. The initial welcome message is excluded from the history to avoid wasting context tokens on system-generated greetings.

9.6 AI Security Measures

The AI integration implements multiple security layers:

- **JWT Authentication:** Only authenticated users can invoke the Edge Function. The JWT is verified by creating a Supabase client with the provided Authorization header and calling `getUser()`.
- **Server-Side Rate Limiting:** An in-memory Map tracks the last request timestamp per user ID, enforcing a minimum three-second interval. This prevents abuse and limits API cost exposure.
- **Memory Cleanup:** When the rate limiter Map exceeds 1000 entries, entries older than 60 seconds are purged to prevent memory exhaustion.
- **Client-Side Cooldown:** A five-second UI-enforced delay between messages provides visual feedback and reduces premature retries.
- **Input Length Validation:** Messages exceeding 2000 characters are rejected with a 400 status code.
- **CORS Restriction:** The Edge Function's CORS headers restrict cross-origin requests to the configured allowed origin (default: `studentdesk.vercel.app`).

- **Timeout Protection:** Each model call uses a 10-second AbortController timeout, preventing indefinite request hanging.

10. Security Implementation

Security in STUDENT DESK is implemented as a multi-layered defense strategy spanning HTTP headers, content policies, input validation, anti-spam mechanisms, URL verification, and error handling. The design philosophy follows the principle of defense-in-depth: no single security mechanism is relied upon exclusively, and multiple independent layers provide overlapping protection.

10.1 HTTP Security Headers

The Vercel deployment configuration (`vercel.json`) defines seven HTTP security headers applied to all responses. These headers instruct the browser to enforce security behaviors that cannot be implemented in JavaScript alone.

Header	Value	Security Purpose
X-Frame-Options	DENY	Prevents the application from being embedded in iframes, blocking clickjacking attacks where an attacker overlays invisible frames to capture user interactions.
X-Content-Type-Options	nosniff	Prevents the browser from MIME-sniffing response content types, ensuring that text files are not interpreted as executable scripts.
Referrer-Policy	strict-origin-when-cross-origin	Controls the Referrer header sent with outbound requests. Same-origin requests include the full path; cross-origin requests include only the origin. This prevents URL path leakage to third-party services.
Permissions-Policy	camera=(), microphone=(), geolocation=()	Explicitly disables browser APIs for camera, microphone, and geolocation access. Even if a dependency or injected script attempts to access these APIs, the browser will deny the request.
Strict-Transport-Security	max-age=63072000; includeSubDomains; preload	Enables HSTS with a two-year duration, forcing all subsequent requests to use HTTPS. The preload directive allows the domain to be included in browser HSTS preload lists, providing protection from the first visit.
X-XSS-Protection	1; mode=block	Activates the browser's built-in XSS filter. While modern browsers have deprecated this header in favor of CSP, it provides an additional layer for older browser versions.

Header	Value	Security Purpose
Content-Security-Policy	(See Section 10.2)	Restricts the sources from which the browser can load resources, preventing code injection and data exfiltration attacks.

10.2 Content Security Policy

The Content Security Policy (CSP) is the most comprehensive browser security mechanism deployed. It defines a whitelist of trusted sources for each resource type:

```
default-src 'self';
script-src 'self';
style-src 'self' 'unsafe-inline';
img-src 'self' data: https://*.supabase.co;
font-src 'self';
connect-src 'self' https://*.supabase.co https://*.supabase.in wss://*.supabase.co;
frame-ancestors 'none';
base-uri 'self';
form-action 'self'
```

Directive Analysis:

- `default-src 'self'` — All resource types not explicitly listed are restricted to the application's own origin.
- `script-src 'self'` — JavaScript can only be loaded from the application's own origin. No inline scripts, eval(), or external script sources are permitted. This is the strongest protection against XSS attacks.
- `style-src 'self' 'unsafe-inline'` — Stylesheets can be loaded from the application's origin, with inline styles permitted. The `unsafe-inline` exception is necessary because Tailwind CSS and CSS-in-JS solutions inject styles at runtime. While this weakens CSS injection protection, it does not enable script execution.
- `img-src 'self' data: https://*.supabase.co` — Images can be loaded from the application origin, data URIs (used for inline SVG icons), and Supabase storage (for any future image assets).
- `connect-src 'self' https://*.supabase.co https://*.supabase.in wss://*.supabase.co` — Network connections (XHR, Fetch, WebSocket) are restricted to the application origin and Supabase domains. The `wss://` protocol allows Supabase Realtime WebSocket connections. The `.supabase.in` domain covers Supabase's alternative infrastructure endpoints.
- `frame-ancestors 'none'` — Complements X-Frame-Options by preventing any site from embedding this application in a frame.
- `base-uri 'self'` — Prevents `<base>` tag injection attacks that could redirect relative URLs to an attacker-controlled origin.
- `form-action 'self'` — Restricts form submission targets to the application's own origin, preventing form hijacking attacks.

10.3 Input Validation Framework

All user inputs are validated using Zod runtime schemas before being processed or submitted to the backend. Zod provides TypeScript-first schema definitions with type inference, meaning the same schema serves as both a runtime validator and a compile-time type definition.

Registration Form Schema:

```
z.object({
  name: z.string().trim().min(1, 'Name is required').max(100, 'Name too long'),
  email: z.string().trim().email('Invalid email').max(255, 'Email too long'),
```

```
  password: z.string().min(8, 'Min 8 characters').max(100),
  confirmPassword: z.string()
}).refine(data => data.password === data.confirmPassword, {
  message: 'Passwords do not match',
  path: ['confirmPassword']
});
```

Contact Form Schema:

```
z.object({
  name: z.string().trim().min(1).max(100),
  email: z.string().trim().email().max(255),
  message: z.string().trim().min(1).max(1000)
});
```

Upload Notes Schema: Title must be at least 3 characters; course, year, semester, and subject selections are required; the uploaded file must be a PDF with a maximum size of 10MB.

Edge Function Validation: The Edge Function independently validates the incoming message for emptiness and length (maximum 2000 characters), and sanitizes the conversation history by slicing to the last six entries. This server-side validation ensures that malicious clients cannot bypass frontend validation.

10.4 Anti-Spam Protection

The contact form implements a three-layer anti-spam system designed to prevent automated form submission without impacting legitimate users:

Layer 1 — Honeypot Field: A hidden input field is rendered in the contact form but made invisible via CSS. Legitimate users never interact with this field, but automated bots frequently fill all form fields indiscriminately. If the honeypot field contains any value when the form is submitted, the submission is silently discarded without providing feedback to the bot:

```
const [honeypot, setHoneypot] = useState('');
// In submit handler:
if (honeypot) return; // Silent rejection
```

Layer 2 — Timing Check: The component records the timestamp when the form is rendered. Submissions that occur within two seconds of page load are rejected, as human users require significantly more time to read and fill the form:

```
const [formLoadedAt] = useState(Date.now());
// In submit handler:
if (Date.now() - formLoadedAt < 2000) {
  toast.error('Please take a moment before submitting.');
  return;
}
```

Layer 3 — Client-Side Rate Limiting: A minimum 30-second interval is enforced between successive contact form submissions from the same browser session:

```
const [lastSubmitAt, setLastSubmitAt] = useState(0);
// In submit handler:
if (Date.now() - lastSubmitAt < 30000) {
  toast.error('Please wait before sending another message.');
  return;
}
```

10.5 URL and Parameter Validation

Download URL Origin Validation: Before initiating a file download, the application validates that the download URL's hostname ends with `.supabase.co` or `.supabase.in`. This prevents a scenario where a compromised or malformed database record could direct the user to download from an attacker-controlled server:

```
const url = new URL(fileUrl);
if (!url.hostname.endsWith('.supabase.co') &&
    !url.hostname.endsWith('.supabase.in')) {
  toast.error('Invalid download URL');
  return;
}
```

Route Parameter UUID Validation: Route parameters such as `courseId` and `subjectId` are validated against a UUID v4 regular expression before being used in database queries. This prevents queries with malformed IDs from reaching the database:

```
const UUID_RE = /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i;
if (!subjectId || !UUID_RE.test(subjectId)) {
  navigate('/dashboard');
  return;
}
```

ILIKE Pattern Injection Prevention: The Dashboard search feature uses ILIKE queries for case-insensitive text matching. To prevent users from injecting wildcard characters that could alter query behavior, the search input is sanitized by escaping `%`, `_`, and `\` characters:

```
const escaped = searchQuery.replace(/[%_\\]/g, '\\$&');
```

10.6 Error Handling Security

ErrorBoundary Component: The application wraps the entire component tree in a React ErrorBoundary class component. When an unhandled rendering error occurs, the boundary catches it and displays a user-friendly error page. In development mode, the actual error message is shown for debugging. In production, error details are hidden to prevent information leakage:

```
{import.meta.env.DEV && this.state.error && (
  <p className="text-sm text-destructive">
    {this.state.error.message}
  </p>
)}
```

Console Log Stripping: Production builds are processed through Terser with `drop_console: true` and `drop_debugger: true` options. This removes all `console.log`, `console.warn`, `console.error`, and `debugger` statements from the production bundle, preventing sensitive information from being exposed in the browser's developer console.

Edge Function Error Responses: The Edge Function returns structured error responses with appropriate HTTP status codes (400, 401, 405, 429, 500). Error messages provided to the client are user-friendly and do not expose stack traces, internal function names, or infrastructure details. Internal errors are logged to the server-side console (accessible only through Supabase's function logs) for debugging purposes.

11. Performance Optimization

Performance optimization in STUDENT DESK targets three dimensions: initial load performance (Time to Interactive), runtime performance (rendering efficiency and data fetching), and perceived performance (user experience during loading states). The optimizations are implemented across the build pipeline, the application code, the database layer, and the network configuration.

11.1 Code Splitting and Lazy Loading

All fifteen page components are loaded on-demand using React's `lazy()` function, which leverages Vite's dynamic import splitting to create a separate JavaScript chunk for each page:

```
const Home = lazy(() => import("./pages/Home"));
const Dashboard = lazy(() => import("./pages/Dashboard"));
const Profile = lazy(() => import("./pages/Profile"));
const CourseYears = lazy(() => import("./pages/CourseYears"));
const YearSemesters = lazy(() => import("./pages/YearSemesters"));
const SemesterSubjects = lazy(() => import("./pages/SemesterSubjects"));
const SubjectNotes = lazy(() => import("./pages/SubjectNotes"));
const AdminPanel = lazy(() => import("./pages/AdminPanel"));
const Contact = lazy(() => import("./pages/Contact"));
const About = lazy(() => import("./pages/About"));
const Register = lazy(() => import("./pages/Register"));
const Login = lazy(() => import("./pages/Login"));
const ForgotPassword = lazy(() => import("./pages/ForgotPassword"));
const UpdatePassword = lazy(() => import("./pages/UpdatePassword"));
const NotFound = lazy(() => import("./pages/NotFound"));
```

When the user navigates to a route, only the JavaScript for that specific page is downloaded and executed. All lazy-loaded components are wrapped in a `<Suspense>` boundary that renders a centered spinner animation while the page chunk is being downloaded:

```
<Suspense fallback={<PageLoader />}>
  <Routes>
    {/* Route definitions */}
  </Routes>
</Suspense>
```

This approach reduces the initial bundle size to only the core application shell (React, Router, contexts, and shared components), with page-specific code loaded progressively as the user navigates.

11.2 Bundle Splitting with Manual Chunks

Vite's Rollup-based build pipeline is configured with manual chunk definitions to create predictable, optimized bundle segments:

```
build: {
  rollupOptions: {
    output: {
      manualChunks: {
        vendor: ["react", "react-dom", "react-router-dom"],
        supabase: ["@supabase/supabase-js"],
        ui: [
          "@radix-ui/react-dialog",
          "@radix-ui/react-dropdown-menu",
```

```
            "@radix-ui/react-tabs",
            "@radix-ui/react-scroll-area",
            "@radix-ui/react-select"
          ],
          utils: ["clsx", "tailwind-merge", "class-variance-authority", "zod"],
          markdown: ["react-markdown"],
        },
      },
    },
}
```

This configuration produces five named chunks:

- **vendor** (~130KB gzipped): Core React libraries that change infrequently. Cached independently from application code.
- **supabase** (~50KB gzipped): The Supabase client library, isolated so that updates to the backend SDK don't invalidate the vendor cache.
- **ui** (~40KB gzipped): Radix UI primitives used by shadcn/ui components. These are heavy dependencies with stable APIs, making them ideal cache candidates.
- **utils** (~15KB gzipped): Small utility libraries (clsx, tailwind-merge, CVA, Zod) grouped together for efficient caching.
- **markdown** (~20KB gzipped): The React Markdown renderer, loaded only when the chatbot is used on SubjectNotes pages.

11.3 Asset Caching Strategy

The Vercel deployment configuration implements a two-tier caching strategy:

Hashed Static Assets (1-year immutable cache):

```
{
  "source": "/assets/(.*)",
  "headers": [
    { "key": "Cache-Control", "value": "public, max-age=31536000, immutable" }
  ]
}
```

All files in the `/assets/` directory are generated by Vite with content-hash filenames (e.g., `index-a1b2c3d4.js`). The `immutable` directive tells browsers and CDN caches that these files will never change—if the content changes, a new filename is generated. This eliminates unnecessary revalidation requests.

Favicon (24-hour cache):

```
{
  "source": "/favicon.png",
  "headers": [
    { "key": "Cache-Control", "value": "public, max-age=86400" }
  ]
}
```

The favicon is cached for 24 hours, allowing branding updates to propagate within a day while avoiding repeated network requests during normal usage.

11.4 Database Query Optimization

Composite Index for Navigation Queries: The most frequently executed query in the application is the SemesterSubjects page query: `WHERE course_id = ? AND year = ? AND semester = ?`. The composite index `idx_subjects_course_year_sem` on `(course_id, year, semester)` allows PostgreSQL to satisfy this three-column filter with a single index scan rather than a sequential table scan.

Trigram Indexes for Text Search: The Dashboard search feature uses `ILIKE` queries with leading wildcards (`%search%`), which cannot use standard B-tree indexes. The `pg_trgm` extension with GIN indexes on `subjects.name` and `subjects.code` enables index-accelerated trigram matching, reducing search query time from sequential scan latency to sub-millisecond index lookup latency.

Debounced Search with AbortController: The Dashboard search input is debounced by 300 milliseconds. Each search creates an AbortController that is aborted when the user types the next character, canceling the in-flight database query and preventing wasted network roundtrips:

```
useEffect(() => {
  const controller = new AbortController();
  const timeoutId = setTimeout(async () => {
    const { data } = await supabase
      .from('subjects')
      .select('*, courses(short_name)')
      .or(`name.ilike.%${escaped}%,code.ilike.%${escaped}%`)
      .limit(5)
      .abortSignal(controller.signal);
  }, 300);
  return () => {
    clearTimeout(timeoutId);
    controller.abort();
  };
}, [searchQuery]);
```

11.5 Network Optimization

Preconnect Hints: The `index.html` includes preconnect and DNS prefetch hints for the Supabase domain:

```
<link rel="preconnect" href="https://wruwdmjcuvtugjshabpa.supabase.co" crossorigin />
<link rel="dns-prefetch" href="https://wruwdmjcuvtugjshabpa.supabase.co" />
```

The `preconnect` hint instructs the browser to establish a TCP connection and perform the TLS handshake with the Supabase server before any API request is made. This eliminates approximately 100–300ms of latency from the first database query. The `dns-prefetch` provides a fallback for older browsers that do not support preconnect, at least resolving the DNS lookup in advance.

11.6 Build Optimization

Terser Minification: Production builds use Terser for JavaScript minification with aggressive dead code elimination:

```
build: {
  sourcemap: false,
  minify: "terser",
  terserOptions: {
    compress: {
      drop_console: true,
      drop_debugger: true,
    },
  },
}
```

- `sourcemap: false` — Source maps are not generated for production builds, reducing build time and preventing source code exposure.
- `drop_console: true` — All console statements are removed, reducing bundle size and preventing information leakage.

- `drop_debugger: true` — All debugger statements are removed, preventing accidental breakpoints in production.

11.7 React Query Caching Layer

The TanStack React Query library is configured at the application root with a centralized QueryClient:

```
const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 1000 * 60 * 5,      // 5 minutes
      gcTime: 1000 * 60 * 30,         // 30 minutes
      retry: 1,                        // Retry failed requests once
      refetchOnWindowFocus: false,     // Disable automatic refetch on focus
    },
  },
});
```

- **staleTime (5 minutes):** Cached data is considered fresh for five minutes. During this period, components that request the same data receive the cached version immediately without triggering a network request. This is particularly effective for the course and subject catalogs, which change infrequently.
- **gcTime (30 minutes):** Unused cached data is garbage collected after thirty minutes, freeing memory for long-running sessions.
- **retry: 1** — Failed requests are retried once before surfacing an error, handling transient network issues gracefully.
- **refetchOnWindowFocus: false** — Automatic background refetching when the user returns to the browser tab is disabled. This prevents unnecessary API calls that would not provide meaningful data updates for academic content.

12. DevOps and CI/CD

The DevOps infrastructure for STUDENT DESK is designed to enforce code quality automatically and minimize deployment friction. The pipeline integrates static analysis, type checking, and build validation into a continuous integration workflow, with deployment handled automatically by the hosting platform.

12.1 Continuous Integration Pipeline

The CI pipeline is implemented as a GitHub Actions workflow that triggers on every push to the `main` branch and every pull request targeting `main`. The workflow executes three sequential validation steps:

```yaml
name: CI
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version-file: '.nvmrc'
          cache: 'npm'
      - run: npm ci
      - run: npm run lint
      - run: npx tsc --noEmit
      - run: npm run build
```

Step 1 — ESLint Linting (`npm run lint`): The ESLint configuration uses the flat config format (`eslint.config.js`) with TypeScript-ESLint rules. This step catches code style violations, unused variables, missing dependencies in hooks, and other static analysis warnings. The build fails if any lint errors are detected.

Step 2 — TypeScript Type Checking (`npx tsc --noEmit`): TypeScript is configured in strict mode via `tsconfig.app.json`, enabling all strict compiler flags including `noImplicitAny`, `strictNullChecks`, `strictFunctionTypes`, `strictBindCallApply`, `strictPropertyInitialization`, and `noUncheckedIndexedAccess`. The `--noEmit` flag ensures that only type checking is performed without generating output files. This step catches type mismatches, missing property assignments, nullable access violations, and other type-level errors.

Step 3 — Production Build (`npm run build`): The Vite production build is executed to verify that the application compiles successfully with all optimizations (Terser minification, manual chunks, code splitting). This step catches import resolution failures, missing environment variables, incompatible dependency versions, and other build-time errors.

12.2 Deployment Pipeline

The deployment pipeline uses two independent channels:

Frontend Deployment (Automated): Vercel is connected to the GitHub repository and automatically deploys the frontend from the `main` branch. When a commit lands on main (either through a direct push or

a merged pull request), Vercel triggers a build pipeline that:

1. Installs dependencies using the lockfile.
2. Executes the Vite production build.
3. Deploys the static output to Vercel's global CDN with the security headers and caching rules defined in `vercel.json`.
4. Provides a unique preview URL for each deployment and assigns the production URL (`studentdesk.vercel.app`) to the latest main branch deployment.

Backend Deployment (Manual): Supabase Edge Functions and database migrations are deployed manually using the Supabase CLI:

```
# Deploy the AI chatbot Edge Function
npx supabase functions deploy ask-gemini

# Apply new database migrations
npx supabase db push
```

Edge Function secrets (GROQ_API_KEY, ALLOWED_ORIGIN) are set via the Supabase CLI:

```
supabase secrets set GROQ_API_KEY=your_key
supabase secrets set ALLOWED_ORIGIN=https://studentdesk.vercel.app
```

12.3 Environment Configuration

The application uses two categories of environment variables:

Frontend Variables (committed as `.env` template):

Variable	Purpose
`VITE_SUPABASE_URL`	The Supabase project URL for database, auth, and storage API calls
`VITE_SUPABASE_PUBLISHABLE_KEY`	The Supabase anonymous/public key for client-side authentication

These variables are prefixed with `VITE_` to ensure Vite exposes them to the client-side bundle via `import.meta.env`.

Edge Function Secrets (server-side only):

Variable	Purpose
`GROQ_API_KEY`	API key for the Groq inference service
`ALLOWED_ORIGIN`	CORS origin whitelist for the Edge Function
`SUPABASE_URL`	Auto-set by Supabase runtime
`SUPABASE_ANON_KEY`	Auto-set by Supabase runtime

12.4 Infrastructure as Code

The database schema is managed through version-controlled SQL migration files in the `supabase/migrations/` directory. Each migration is timestamped and applied sequentially:

Migration	Purpose
`20251125145915_*.sql`	Initial schema: tables, RLS policies, triggers, seed data (6 courses)

Migration	Purpose
`20251125145926_*.sql`	Security fix: search_path pinning for security definer functions
`20251125161931_*.sql`	RLS policy tightening for stricter access control
`20251125163402_*.sql`	Resource type enumeration: notes vs. question papers distinction
`20251125180105_*.sql`	Contact messages table: public insert, admin-only management
`20260215000000_*.sql`	Scalability indexes: composite, FK, trigram GIN indexes
`20260216000000_*.sql`	User trigger fix: branch extraction, text length constraints

This migration-based approach ensures that database changes are reproducible, auditable, and can be applied to new environments by running `supabase db push`.

13. Features and Modules

13.1 Public Pages

The following pages are accessible without authentication, providing information about the platform and enabling user onboarding:

Home Page (`/`): The landing page presents the platform's value proposition through a hero section with gradient text styling, a feature grid highlighting key capabilities (organized notes, AI chatbot, exam-type tags, 24/7 access), a benefits section, and a call-to-action button directing users to registration. The Home page includes JSON-LD structured data (`WebSite` schema with `SearchAction`) for enhanced search engine indexing.

About Page (`/about`): Presents the platform's mission statement, a description of what STUDENT DESK offers, and community-oriented messaging. Styled with card-based layout and gradient accents.

Contact Page (`/contact`): A validated contact form with Zod schema enforcement, honeypot anti-spam field, timing-based bot detection, and client-side rate limiting. Form submissions are inserted into the `contact_messages` table via Supabase.

Login Page (`/login`): Email and password authentication form with validation feedback. Includes a link to the Forgot Password page and a registration prompt for new users.

Register Page (`/register`): Sign-up form with fields for name, email, password, password confirmation, and optional engineering branch selection. Validated against a Zod schema with real-time error messaging.

Forgot Password Page (`/forgot-password`): Password reset request form that sends a reset email via Supabase Auth with a redirect URL pointing to the Update Password page.

Update Password Page (`/update-password`): Accessible via the password reset email link. Allows the user to set a new password. The AuthContext listens for the `PASSWORD_RECOVERY` auth event and navigates to this page automatically.

13.2 Protected Pages

Dashboard (`/dashboard`): The primary navigation hub for authenticated users. Displays all available engineering courses as cards with course names, abbreviations, and descriptions. Features a prominent search bar with gradient glow effect for instant subject search. The search uses debounced ILIKE queries with AbortController and displays results in a dropdown overlay. Admin users see an additional "Admin Panel" access card.

Profile (`/profile`): User profile management page displaying account information (email, registration date) and editable fields (name, engineering branch). Profile updates are written to the profiles table with automatic `updated_at` timestamp management via the database trigger.

13.3 Academic Browse Hierarchy

The browse pages implement the five-level academic navigation hierarchy:

CourseYears (`/course/:courseId`): Displays four year cards (Year 1 through Year 4) for the selected course. The course name is fetched from the database and displayed as the page heading.

YearSemesters (`/course/:courseId/year/:year`): Displays two semester cards (Semester 1 and Semester 2) for the selected course and year combination.

SemesterSubjects (`/course/:courseId/year/:year/semester/:semester`): Lists all subjects for the selected course, year, and semester. Each subject card displays the subject name and code, with navigation to the SubjectNotes page.

SubjectNotes (`/course/:courseId/year/:year/semester/:semester/subject/:subjectId`): The content delivery page. Displays all available notes and question papers for the selected subject, organized as cards with title, description, resource type badge, exam type badge, and View/Download action buttons. The AI ChatBot floating widget is rendered on this page, providing subject-specific doubt resolution.

Each Note/Question Paper card includes: - Title and optional description - Resource type badge (Notes or Question Papers) - Exam type badge (Regular, Supply, or Both) with color-coded styling - View button (opens PDF in a new browser tab) - Download button (fetches the PDF blob and triggers a local download)

13.4 Admin Panel

The Admin Panel (`/admin`) is a tabbed interface providing four content management sections:

Upload Notes Tab: A comprehensive upload form allowing administrators to: 1. Select a target course from a dropdown populated by the courses table. 2. Select the academic year (1–4) and semester (1–2). 3. Select the target subject from a dropdown filtered by the selected course, year, and semester. 4. Choose the resource type (Notes or Question Papers). 5. Choose the exam type (Regular, Supply, or Both). 6. Enter a title and optional description. 7. Select a PDF file (maximum 10MB, validated for MIME type). 8. Submit the upload, which first uploads the file to Supabase Storage and then creates a metadata record in the notes table.

Manage Subjects Tab: A full CRUD interface for academic subjects. Administrators can add new subjects with course, year, semester, name, and code fields. Existing subjects can be edited or deleted. The subject listing updates in real-time after each operation.

Manage Courses Tab: A full CRUD interface for engineering courses (branches). Administrators can add new courses with full name, short name, and description. Existing courses can be edited or deleted.

Messages Tab: A management interface for contact form submissions. Displays messages with sender name, email, message content, and submission timestamp. Administrators can toggle the read/unread status, delete individual messages, and navigate through paginated results (20 messages per page, ordered by creation date descending).

13.5 Dark Mode System

The application implements a three-mode theme system managed by the ThemeContext:

- **Light Mode:** Forces the light color scheme regardless of system preference.
- **Dark Mode:** Forces the dark color scheme regardless of system preference.
- **System Mode:** Automatically matches the operating system's color preference by listening to the `prefers-color-scheme` media query.

The selected theme is persisted to `localStorage` and restored on subsequent visits. When in System mode, the context registers a `MediaQueryList` change listener that updates the resolved theme in real-time if the user changes their OS preference.

Theme switching is implemented by toggling the `light` or `dark` class on the document root element (`<html>`), which Tailwind CSS uses to apply the appropriate color palette. The toggle button in the Navbar displays a Sun icon in dark mode and a Moon icon in light mode, with an `aria-label` for accessibility.

13.6 SEO and Accessibility

Search Engine Optimization:

Every page renders dynamic meta tags via the `PageMeta` component (built on `react-helmet-async`), including:

- `<title>` with page-specific content and "STUDENT DESK" brand suffix
- `<meta name="description">` with page-specific descriptions
- `<link rel="canonical">` with the full canonical URL

- Open Graph tags (`og:title`, `og:description`, `og:url`, `og:type`, `og:image`)
- Twitter Card tags (`twitter:card`, `twitter:title`, `twitter:description`, `twitter:image`)

The Home page includes JSON-LD structured data implementing the `WebSite` schema with a `SearchAction`, enabling potential sitelinks search box integration in Google search results.

Static fallback meta tags are defined in `index.html` for search engine crawlers that do not execute JavaScript, ensuring basic SEO indexing coverage even without client-side rendering.

A `robots.txt` file at the application root allows all crawlers unrestricted access to the public pages.

Accessibility (a11y):

- **Skip to Content Link:** A visually hidden "Skip to main content" link is rendered as the first focusable element in the DOM. It becomes visible on Tab key focus, allowing keyboard users to bypass the navigation bar.
- **ARIA Labels:** All icon-only buttons (chatbot toggle, menu button, theme toggle) include descriptive `aria-label` attributes. The `<nav>` element includes `aria-label="Main navigation"`.
- **Semantic HTML:** The application uses semantic HTML5 landmarks: `<nav>` for navigation, `<main id="main-content">` for primary content, and `<footer>` for the site footer. Heading hierarchy follows a logical structure (h1 $\rightarrow$ h2 $\rightarrow$ h3) on every page.
- **Form Labels:** All form inputs are associated with `<Label>` elements using the `htmlFor` attribute, ensuring screen reader compatibility.
- **Keyboard Navigation:** The chatbot can be closed with the Escape key. All interactive elements are focusable and activatable via keyboard. The Sheet component (mobile menu) supports keyboard-driven open/close.

14. Testing and Validation

14.1 Static Analysis

The project uses two primary static analysis tools:

ESLint: Configured via `eslint.config.js` using the flat config format with TypeScript-ESLint integration. The ruleset includes React Hooks lint rules (enforcing rules of hooks and exhaustive dependencies) and React Refresh compatibility rules (ensuring components are exportable via HMR). ESLint runs on every CI pipeline execution and as part of the local development workflow.

TypeScript Strict Mode: The `tsconfig.app.json` enables TypeScript's strict compiler family, which activates:

- `strict: true` — Enables all strict type-checking options as a group.
- `noImplicitAny` — Every variable must have an explicit or inferable type.
- `strictNullChecks` — Nullable types must be explicitly handled.
- `strictFunctionTypes` — Function parameter types are checked contravariantly.
- `noUnusedLocals` and `noUnusedParameters` — Unused declarations are flagged as errors.

This configuration catches a broad range of potential runtime errors at compile time, including null reference exceptions, type mismatches in function calls, and unhandled optional properties.

14.2 Build Validation

The CI pipeline executes a full production build (`npm run build`) on every push and pull request. This step validates:

- All imports resolve to existing modules.
- All TypeScript files compile without errors.
- Vite's Rollup bundler can successfully split and minify all chunks.
- Manual chunk definitions reference existing dependencies.
- Environment variable references use the correct `VITE_` prefix.

A failing build blocks the pull request from being merged, ensuring that only buildable code reaches the main branch.

14.3 Manual Testing Procedures

The following manual test scenarios are verified before each release:

Authentication Flow Testing: 1. Register a new account with valid credentials → Verify profile creation and role assignment. 2. Attempt registration with an existing email → Verify duplicate detection error. 3. Sign in with valid credentials → Verify session establishment and admin status check. 4. Sign in with invalid credentials → Verify error messaging. 5. Reset password → Verify email delivery and password update flow. 6. Sign out → Verify session clearance and state reset.

Course Navigation Testing: 1. Navigate Dashboard → Course → Year → Semester → Subject → Notes. 2. Verify breadcrumb context at each navigation level. 3. Download a PDF note → Verify file integrity. 4. View a PDF note → Verify new tab rendering. 5. Test the subject search with known and unknown queries.

AI Chatbot Testing: 1. Open the chatbot on a SubjectNotes page → Verify welcome message with subject name. 2. Send a simple greeting → Verify brief, friendly response. 3. Ask a subject-related question → Verify formatted academic response. 4. Request a detailed explanation → Verify comprehensive response. 5. Send a follow-up question → Verify context retention from previous messages. 6. Test the 5-second cooldown → Verify input disabling. 7. Close the chatbot with Escape → Verify panel dismissal.

Admin Panel Testing: 1. Upload a PDF note with all metadata → Verify storage upload and database insertion. 2. Attempt to upload a non-PDF file → Verify rejection. 3. Attempt to upload a file exceeding 10MB → Verify size validation. 4. Add, edit, and delete a subject → Verify CRUD operations. 5. Add, edit, and delete a course → Verify cascade behavior. 6. View, mark as read, and delete contact messages → Verify admin operations.

Security Testing: 1. Attempt to access `/admin` as a non-admin user → Verify redirect to Dashboard. 2. Attempt to access `/dashboard` as an unauthenticated user → Verify redirect to Login. 3. Open browser developer tools → Verify no console.log output in production. 4. Inspect HTTP response headers → Verify all seven security headers are present. 5. Submit the contact form with the honeypot field filled → Verify silent rejection. 6. Submit the contact form within two seconds of page load → Verify timing rejection.

14.4 Security Testing

Content Security Policy Validation: The CSP header is tested by attempting to inject inline scripts, load external scripts, and embed the application in an iframe. All three scenarios are blocked by the configured policy.

CORS Testing: The Edge Function CORS configuration is tested by making cross-origin requests from unauthorized domains. Only requests from the configured `ALLOWED_ORIGIN` are accepted.

Rate Limit Testing: The chatbot rate limiting is tested by sending rapid sequential requests. Both the client-side 5-second cooldown and the server-side 3-second rate limiter are verified to reject premature requests.

14.5 Planned Automated Testing

The following automated testing infrastructure is planned for future implementation:

- **Unit Tests:** Vitest for testing utility functions, context providers, and component logic in isolation.
- **Integration Tests:** Testing Supabase query behavior with mock data to verify database interaction patterns.
- **End-to-End Tests:** Playwright for browser-based testing of complete user flows including registration, navigation, chatbot interaction, and admin operations.
- **Visual Regression Tests:** Component screenshot comparison to detect unintended UI changes.

15. Future Enhancements

The following enhancements are planned for future releases, prioritized by impact and implementation complexity:

High Priority

Automated Test Suite: Implementation of Vitest for unit testing and Playwright for end-to-end testing. This will provide regression detection for all critical user flows and enable confident refactoring of core modules. Test coverage targets: 80% for utility functions, 60% for components, and full E2E coverage for authentication and navigation flows.

React Query Migration: Migration of all manual `useState`/`useEffect` data fetching patterns to React Query `useQuery` and `useMutation` hooks. This will provide automatic cache invalidation, optimistic updates for admin operations, infinite scrolling for large note collections, and background refetching for stale data.

Medium Priority

Progressive Web Application (PWA) Support: Addition of a service worker for offline caching of previously downloaded notes. This would enable students to access their study materials without an internet connection, which is particularly valuable in areas with unreliable connectivity.

Notification System: Email and/or push notifications when new notes are uploaded for subjects that a student has previously accessed. This keeps students informed of new materials without requiring them to manually check the platform.

Analytics Dashboard: A dashboard for administrators showing download counts per subject, popular search queries, active user metrics, and peak usage times. This data would inform content prioritization and platform development decisions.

Student Contributions: A feature allowing students to upload their own notes, with an admin moderation queue to ensure content quality before publication. This would accelerate content growth and foster a community-driven resource ecosystem.

Low Priority

Dynamic Sitemap Generation: Automatic generation of `sitemap.xml` based on the current course, subject, and notes catalog. This would improve search engine indexing of deep-linked academic content pages.

Social Authentication: Integration of Google and GitHub OAuth providers via Supabase Auth. This would reduce registration friction for users who prefer social sign-in over email/password authentication.

In-Browser PDF Viewer: Embedding a PDF viewer component (such as PDF.js) to allow students to read notes within the application without opening a new browser tab. This would improve the user experience for quick content review.

Multi-Language User Interface: Internationalization (i18n) of the application UI to support Hindi, Telugu, and other regional languages, making the platform more accessible to students who are more comfortable with non-English interfaces.

16. Conclusion

STUDENT DESK Version 2.0 represents a comprehensive engineering effort to address the real-world challenge of fragmented academic resource management in the Indian engineering education system. The platform is not a prototype or proof-of-concept—it is a production application deployed on live infrastructure, serving actual users, and processing real academic content.

The technical architecture demonstrates proficiency across the full spectrum of modern web application development:

Frontend Engineering: The React 18 frontend is built with TypeScript strict mode, eliminating entire categories of runtime errors through compile-time type safety. The component architecture leverages the Context API for global state management, React.lazy() for route-level code splitting, and React Hook Form with Zod for performant, schema-validated form handling. The UI is built on shadcn/ui components with Radix UI accessibility primitives, ensuring that the interface is both visually refined and screen-reader compatible.

Backend Engineering: The serverless backend on Supabase Cloud uses PostgreSQL with Row Level Security as the primary data access control mechanism. Every database operation is authorized at the database layer, not the application layer, providing security guarantees that are independent of client implementation. Database performance is optimized through composite indexes, FK indexes, and trigram GIN indexes for text search.

AI/ML Integration: The AI chatbot system demonstrates practical LLM integration with a three-model fallback cascade, conversation memory, context-aware prompting, and multi-layer rate limiting. The system prompt engineering follows a nine-rule behavior specification that balances academic rigor with conversational naturalness.

Security Engineering: The security posture extends across seven HTTP headers (including HSTS with two-year preload, strict CSP, and clickjacking protection), database-level RLS policies, server-side and client-side rate limiting, Zod input validation, honeypot anti-spam mechanisms, URL origin verification, UUID parameter validation, ILIKE injection prevention, console log stripping, and environment-conditional error display.

DevOps Engineering: The delivery pipeline integrates GitHub Actions CI (lint → typecheck → build), automatic Vercel deployments, version-controlled SQL migrations, and environment-separated secret management, ensuring code quality enforcement and reproducible deployments.

The platform currently supports six engineering branches, four academic years, two semesters per year, and an extensible subject and notes catalog. Its architecture is designed for horizontal scalability—the serverless compute model scales automatically with demand, the database indexes are designed for the anticipated growth in content volume, and the CDN-based frontend delivery handles geographic distribution without operational intervention.

STUDENT DESK serves as both a functional academic tool for its target user base and a demonstration of production-quality full-stack engineering in a modern JavaScript ecosystem.

Technical Documentation — STUDENT DESK Version 2.0 Author: Vivek Boga February 2026